

Tax Fraud Detection

Model Development Report

Data cleaning → EDA → 3 rounds of iterative model experimentation → final evaluation

Final recommended model:

XGBoost — threshold 0.338

Test PR-AUC 0.834 · Test F1 0.752 · Precision 0.832 · Recall 0.685

Executive Summary

The dataset contains 50,500 Egyptian taxpayer records (25 columns: business profile, financial figures, tax reporting, compliance history, and behavioral red flags) with 10.7% confirmed/suspected fraud. After cleaning (500 exact duplicate rows removed, ~3% of rows with jointly-missing fields imputed) the working dataset is 50,000 rows at the same 10.72% fraud rate.

Three rounds of modeling experiments (“batches”) were run, each building on the previous one’s evidence rather than assumption:

- Batch 1 — baseline. 9 models compared with correct encoding and model-specific scaling, no imbalance handling. XGBoost, LightGBM, and CatBoost tied at ~0.80 PR-AUC; CatBoost was 22-28x slower to train for no real accuracy gain and was dropped from further tuning.
- Batch 2 — imbalance + redundancy (negative result, reported honestly). Every sampling/class-weighting technique tested (SMOTE, over/under-sampling, combined methods, class weights) increased recall but reduced PR-AUC for every model — none beat the no-sampling baseline. VIF confirmed severe multicollinearity in the financial column block; removing it made no difference for 4 of 5 models and hurt Random Forest.
- Batch 3 — threshold tuning (the deciding result). Since PR-AUC is threshold-independent, tuning the decision threshold on the existing baseline models recovered the same kind of recall gains Batch 2’s sampling chased — without retraining and without sampling’s precision collapse. At comparable recall, threshold tuning beat every sampling technique by 17-19 percentage points of precision, for every model.

Final result: XGBoost, evaluated once on the untouched test set at its threshold of 0.338, achieves 0.834 PR-AUC, 83.2% precision and 68.5% recall (F1 = 0.752) — the best result found anywhere in the project. Test performance was slightly better than validation, confirming the 3-batch decision process did not overfit to validation.

Data & Preprocessing

Dataset

- Raw: 50,500 rows x 25 columns, fraud rate 0.1073
- 4,524 missing cells across Business_Type/Region/Cash_Transactions_Percentage (~3% of rows) — missingness was unrelated to fraud (10.1% vs 10.7% fraud rate with/without), so rows were filled rather than dropped (dropping would have discarded ~150 real fraud cases from an already-rare class).
- 500 fully duplicated rows removed.
- Cleaned: 50,000 rows x 25 columns, fraud rate 0.1072.
- Outliers (IQR rule) were inspected, not blindly removed: Tax_Gap outliers had a 46% fraud rate vs 7% for normal values — they are the fraud signal itself, not data errors, and were kept.

Train / Validation / Test Split

- One fixed, stratified 70/15/15 split (35,000 / 7,500 / 7,500 rows), created once and reused by every batch — 10.72% fraud rate in all three splits.
- The test set was touched exactly once, at the very end of the project, for final evaluation only. Every batch-to-batch decision used training/CV and validation performance.

Encoding & Scaling

- Business_Type, Region → one-hot encoding (nominal, no order).
- Industry_Risk → ordinal encoding with explicit order Low < Medium < High (not LabelEncoder, which would assign an arbitrary alphabetical order).
- CatBoost used its native categorical handling instead of one-hot (evaluated in Batch 1; not carried forward, see Batch 1 section).
- StandardScaler vs RobustScaler was compared for Logistic Regression, SVM, and MLP; RobustScaler won by a small margin and was used from Batch 1 onward. Tree/boosting models (Decision Tree, Random Forest, XGBoost, LightGBM) and Naive Bayes were left unscaled, since scaling doesn't affect them.

Feature Selection & Multicollinearity

EDA found several financial columns are exact mathematical duplicates of each other (e.g. $\text{VAT_Collected} = 14\% \times \text{Annual_Revenue}$; $\text{Expected_Tax} = 22.5\% \times \text{Taxable_Income}$; $\text{Tax_Gap} = \text{Expected_Tax} - \text{Declared_Tax}$). A VIF analysis in Batch 2 confirmed this formally — several columns showed effectively infinite VIF:

```
Profit_Margin: inf | Annual_Revenue: inf | Annual_Expenses: inf | Expense_Ratio: inf |  
Net_Profit: inf | VAT_Collected: inf
```

Behavioral/categorical features (Previous_Violations, Invoice_Mismatch, Missing_Documents, Cash_Transactions_Percentage, Industry_Risk, etc.) all had $\text{VIF} \approx 1.0$ — completely independent, and kept unconditionally per the project plan.

Two reduced feature sets were tested (Batch 2, Track B):

- reduced_v1: dropped the 7 exactly-redundant columns (VAT_Collected, VAT_Paid, Expected_Tax, Taxable_Income, Net_Profit, Expense_Ratio, Profit_Margin) — VIF for Annual_Revenue/Annual_Expenses/Declared_Tax remained severe (hundreds to thousands), because their relationship, while not an exact formula, is still strongly statistical for the ~89% of taxpayers who are compliant.
- reduced_v2: also dropped Annual_Revenue and Annual_Expenses (near-zero standalone fraud correlation in EDA, near-zero feature importance in Batch 1) — fully resolved the multicollinearity, max VIF 1.76.

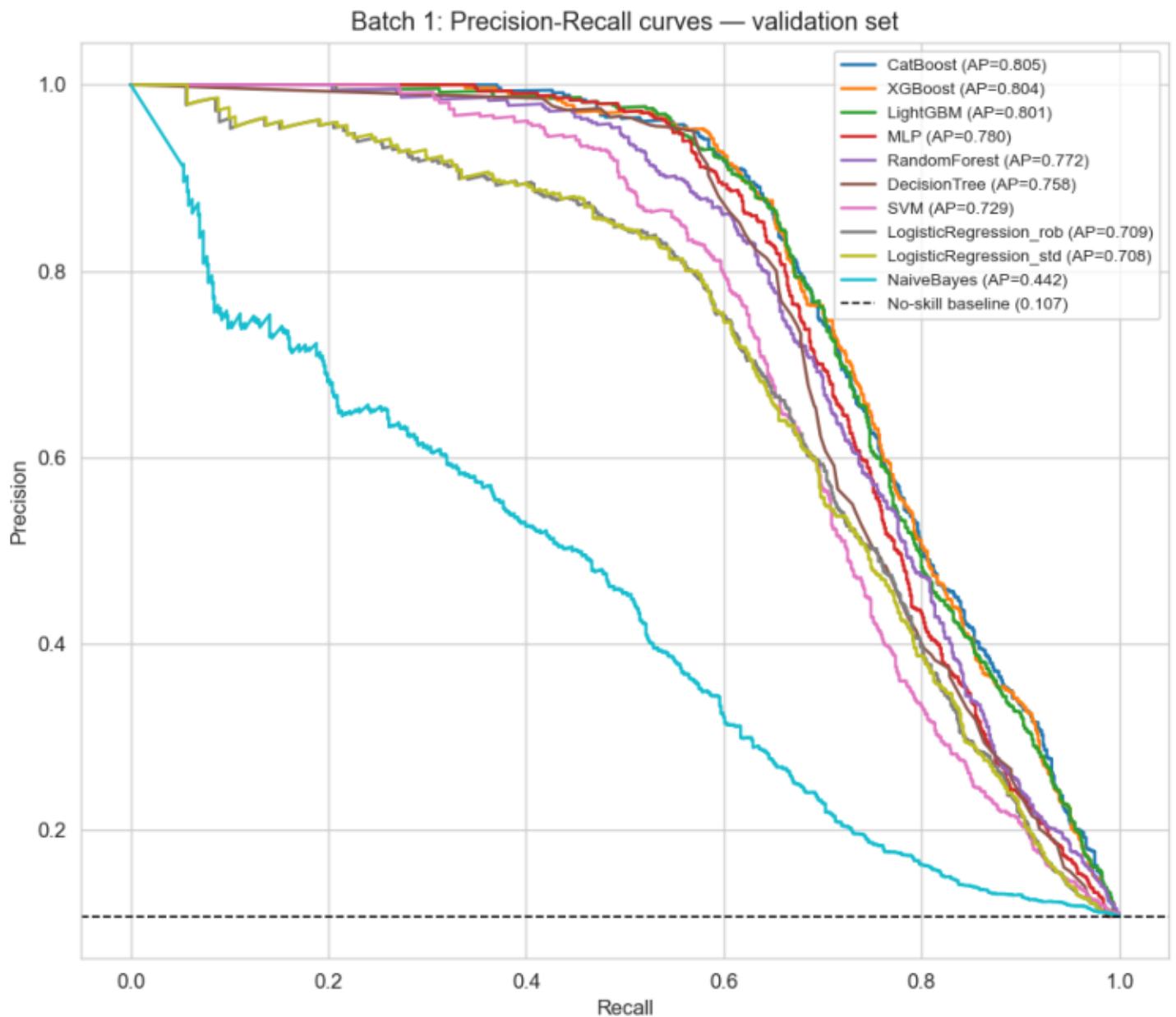
Result: redundancy removal made no meaningful difference for XGBoost, LightGBM, MLP, or Logistic Regression (whose L1 penalty was already implicitly zeroing out redundant coefficients) — but it hurt Random Forest (PR-AUC -0.018). The full feature set was kept for the final model.

Batch 1 — Baseline Results

All 9 required models, correct encoding, model-specific scaling, no imbalance handling, minimal feature removal (only Taxpayer_ID dropped). Ranked by validation PR-AUC, the primary metric given the 10.7%/89.3% class imbalance.

Model	Features	Val PR-AUC	Precision	Recall	F1	Train time
CatBoost	catboost_native	0.8051	0.918	0.600	0.725	1519s
XGBoost	tree	0.8039	0.908	0.613	0.732	55s
LightGBM	tree	0.8008	0.901	0.614	0.731	69s
MLP	rob	0.7804	0.893	0.603	0.720	22s
RandomForest	tree	0.7724	0.930	0.514	0.662	426s
DecisionTree	tree	0.7585	0.926	0.576	0.710	26s
SVM	rob	0.7288	0.864	0.546	0.669	18s
LogisticRegression_rob	rob	0.7087	0.877	0.437	0.583	30s
LogisticRegression_std	std	0.7076	0.878	0.440	0.587	34s
NaiveBayes	tree	0.4419	0.623	0.280	0.386	2s

Batch 1 — Precision-Recall Curves (Validation Set)



CatBoost, XGBoost, and LightGBM are statistically tied (within each other's CV noise).

CatBoost was dropped from further tuning: 1,519s to train vs 55s (XGBoost) / 69s (LightGBM)

for no real accuracy advantage. Naive Bayes is far behind (features are skewed and correlated, violating its independence/Gaussian assumptions).

Batch 2 — Imbalance Handling & Redundancy

Every technique was applied only within training folds (via imblearn pipelines) — never to validation or test data — using each model's Batch 1 hyperparameters held fixed, to isolate the effect of the sampling technique alone.

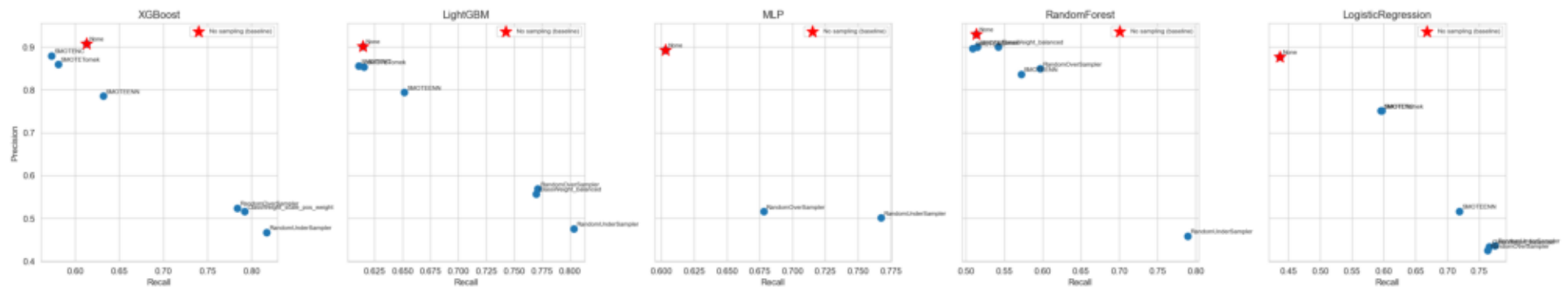
Techniques tested: class weighting, SMOTENC, random over-sampling, random under-sampling, SMOTEENN, SMOTETomek.

Result — reported honestly, including the negative finding:

- Recall improved substantially for every model (e.g. XGBoost: 0.613 → 0.817 with RandomUnderSampler).
- Precision dropped severely in proportion (XGBoost: 0.908 → 0.467 at that same config) — worse than a coin flip on positive predictions.
- PR-AUC decreased for every single technique, on every single model — confirmed programmatically, not just by inspection. The no-sampling baseline was the best configuration for all 5 models tested (LightGBM, XGBoost, Random Forest, MLP, Logistic Regression).
- Conclusion: sampling techniques relocate the operating point on the precision-recall curve; they do not improve the underlying ranking quality PR-AUC measures. This directly motivated Batch 3's threshold-tuning approach as a cleaner alternative.

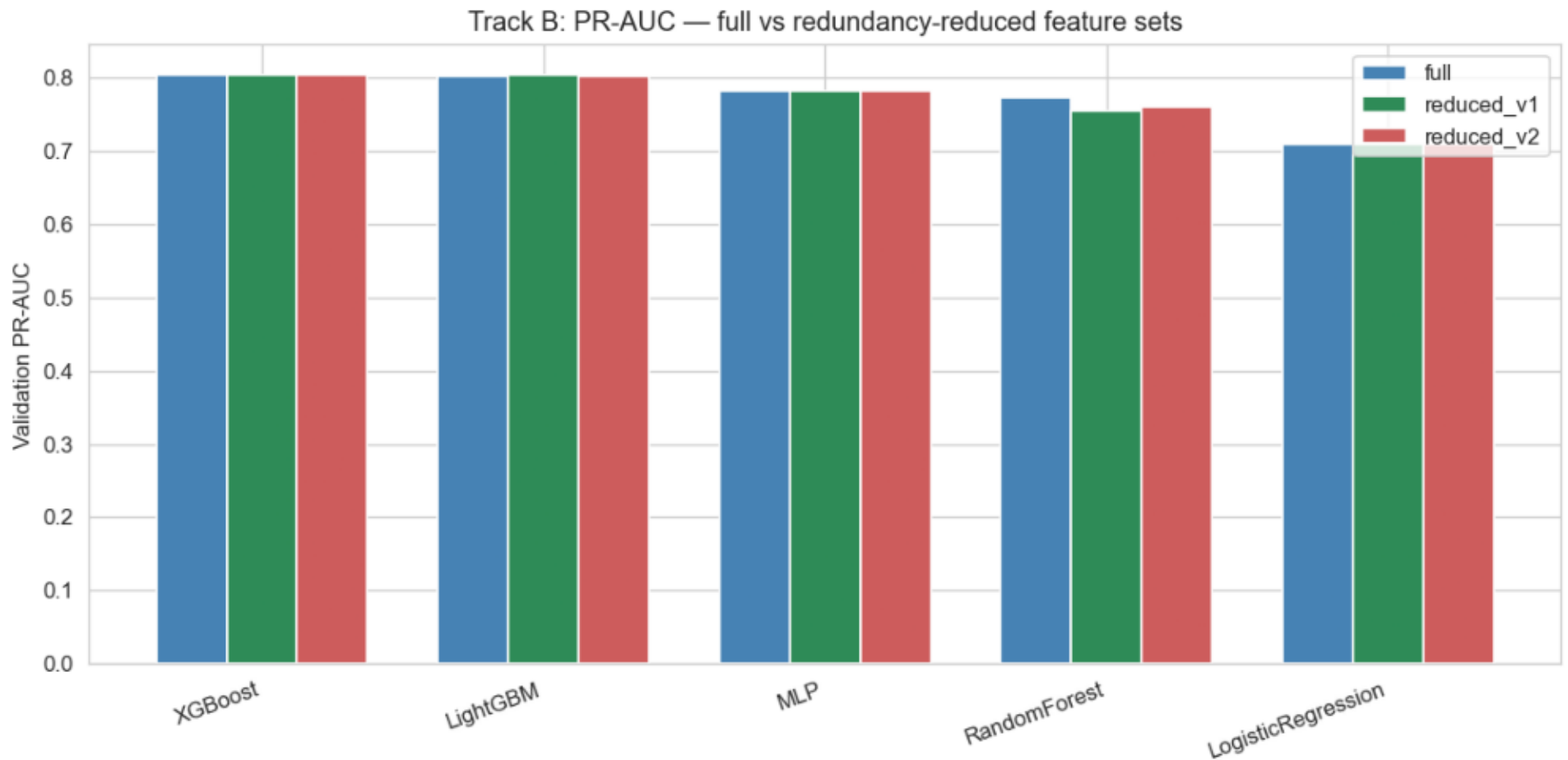
Batch 2 — Precision vs Recall by Sampling Technique

Track A: Precision vs Recall trade-off by sampling technique



Red star = no-sampling baseline. Every technique moves the operating point down and to the right (more recall, much less precision) — none improve the model itself.

Batch 2 — Full vs Redundancy-Reduced Feature Sets



Differences are noise-level (± 0.002 - 0.003 PR-AUC) for XGBoost/LightGBM/Logistic Regression/MLP. Random Forest is the exception: removing the redundant financial columns cost it real signal (-0.018 PR-AUC for reduced_v1).

Batch 3 — Regularization & Threshold Tuning

Part A — Regularization refinement

A narrow re-search around Batch 1's winning configurations for XGBoost, LightGBM, and Logistic Regression, adding L1/L2 regularization strength (reg_alpha/reg_lambda for the boosters, a finer C grid for Logistic Regression). All three improved CV PR-AUC, but by margins too small to matter practically (+0.0003, +0.0002, +0.0007) — Batch 1's exploratory search had already found close to the achievable ceiling.

Part B — Threshold tuning (the deciding result)

Precision and recall trade off against each other for any fixed model, so “maximize both” means finding the best balance — the F1-optimal threshold (F1 is the harmonic mean of precision and recall). This threshold was chosen using out-of-fold cross-validated probabilities on the training set only, then confirmed once on validation — never chosen by looking at validation directly.

Every model's default 0.5 threshold was too conservative. Moving to the F1-optimal threshold (0.23-0.38 depending on model) improved F1 for all 5 finalists, most for Logistic Regression (0.576 → 0.669) and Random Forest (0.662 → 0.712).

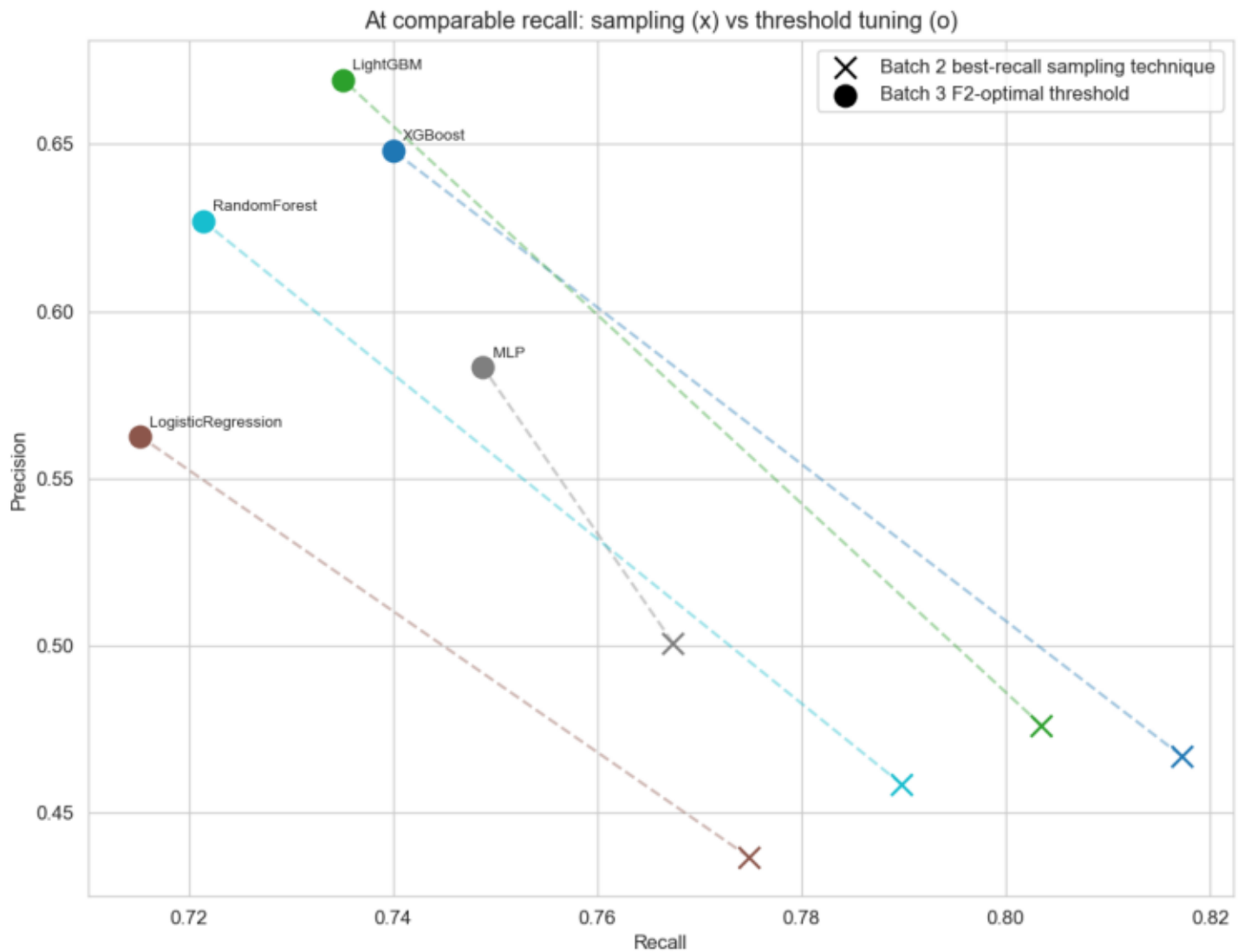
At comparable recall, threshold tuning beat every Batch 2 sampling technique, for every model, by 17-19 percentage points of precision — with zero retraining, since PR-AUC (and the ranking it measures) cannot be changed by moving a threshold.

Batch 3 — F1-Optimal Threshold Per Model (Validation)

XGBoost and LightGBM lead; both carried forward to the final test evaluation.

Model	Threshold	Precision	Recall	F1
XGBoost	0.338	0.845	0.659	0.741
LightGBM	0.378	0.860	0.643	0.736
MLP	0.347	0.820	0.653	0.727
RandomForest	0.339	0.852	0.611	0.712
LogisticRegression	0.235	0.746	0.607	0.669

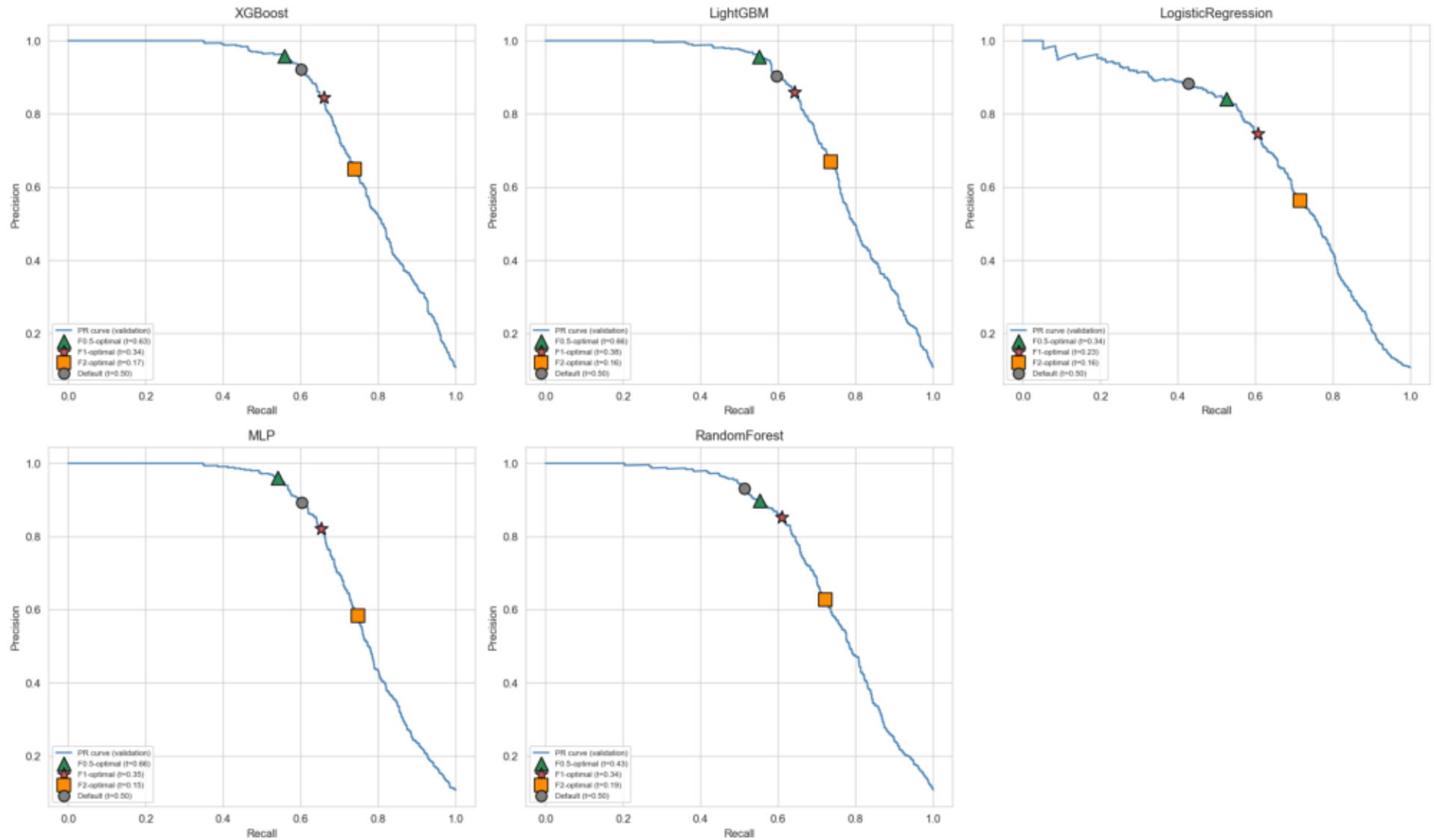
Batch 3 — Threshold Tuning vs Sampling (Batch 2)



X = Batch 2's best-recall sampling technique. O = Batch 3's F2-optimal threshold. At similar recall, every model's threshold-tuned precision is far above its sampling-based precision.

Batch 3 — Precision-Recall Curves With Tuned Thresholds

Batch 3: Precision-Recall curves with tuned thresholds (validation set)



Final Test-Set Evaluation

The test set (7,500 rows, 10.72% fraud, never used in any prior decision) was evaluated exactly once, for the two finalists from Batch 3.

XGBoost: Validation PR-AUC 0.8037 → Test PR-AUC 0.8335 (+0.0298)

LightGBM: Validation PR-AUC 0.7994 → Test PR-AUC 0.8322 (+0.0327)

Test performance was slightly higher than validation for both models — the opposite of what overfitting to validation across 3 batches of decisions would look like.

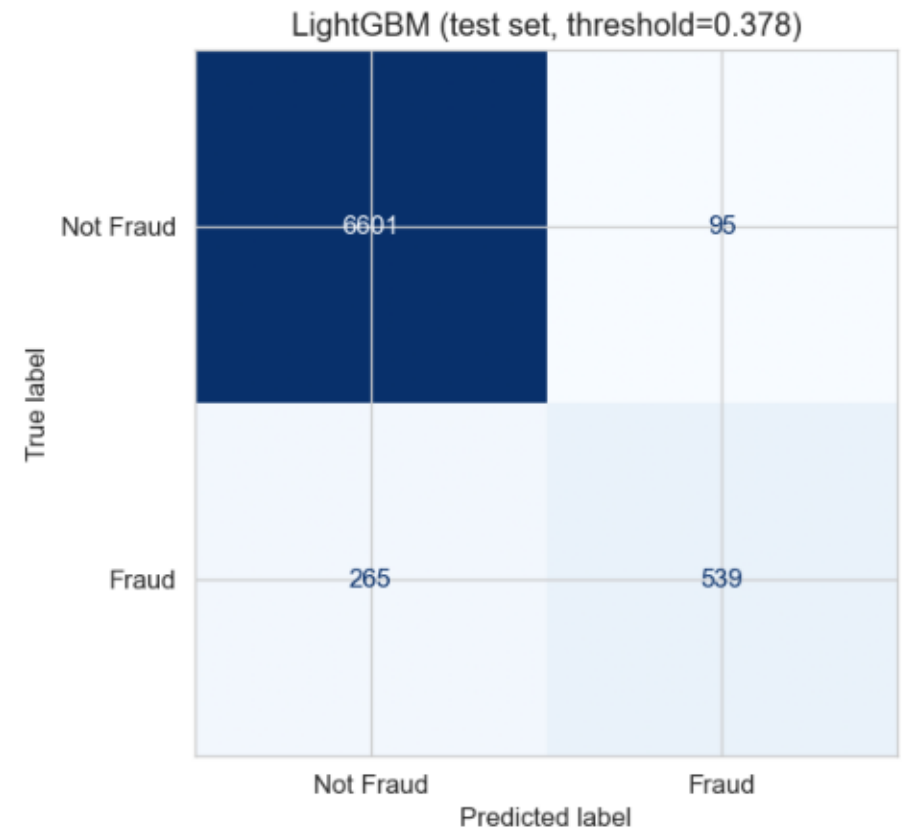
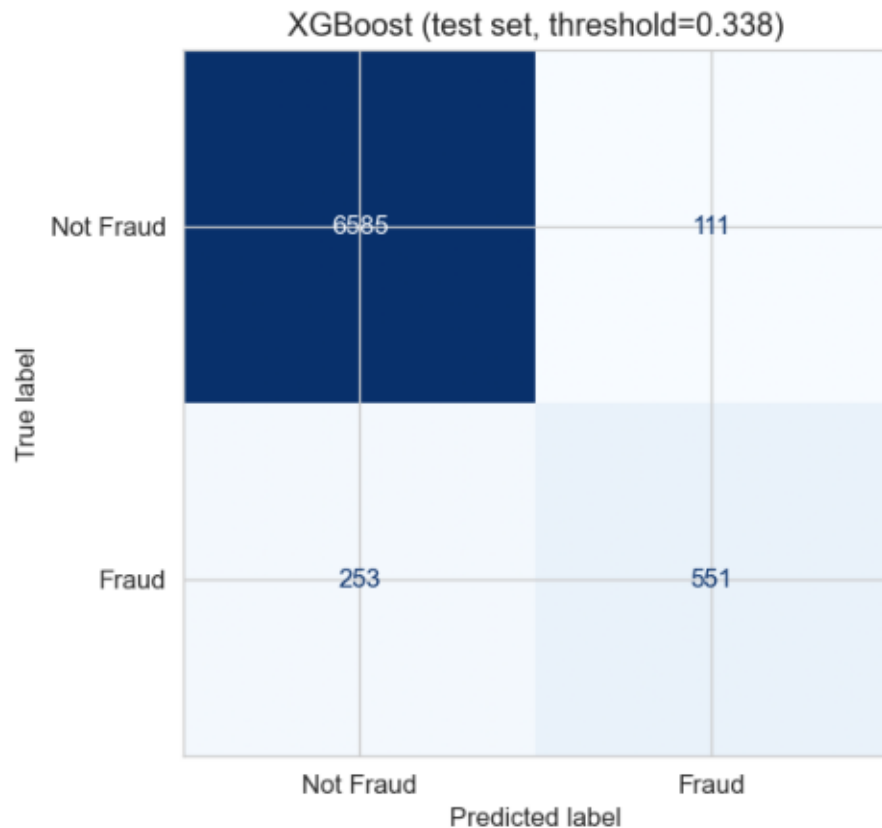
This is strong evidence the whole train/validate/test discipline held up.

Test-set performance at tuned thresholds:

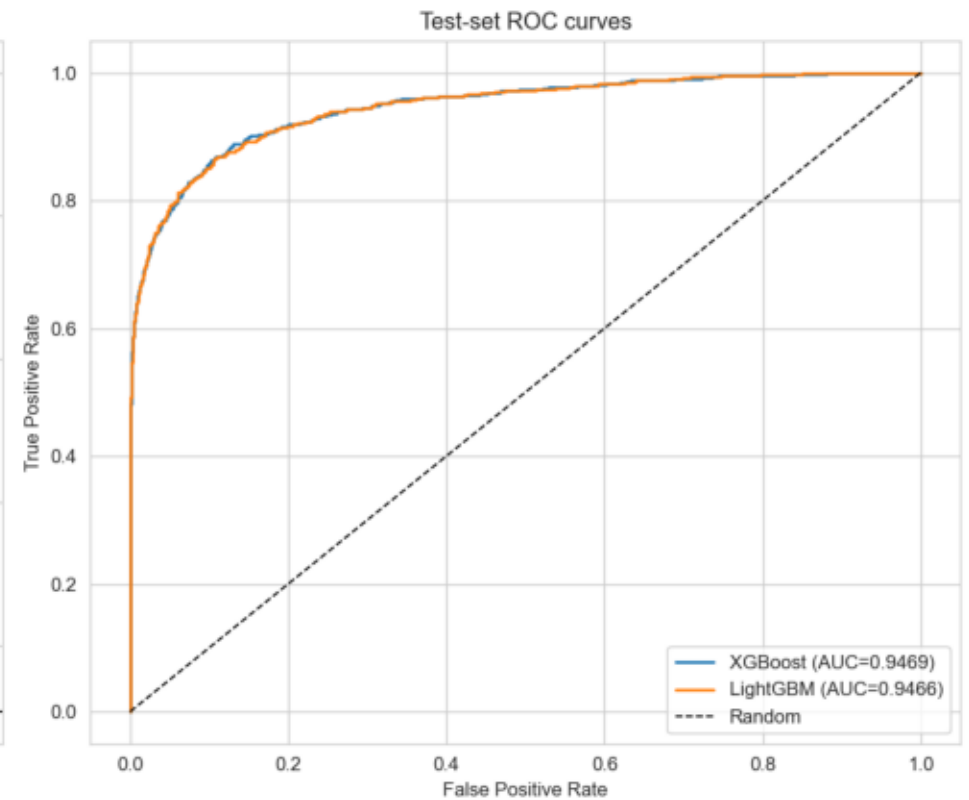
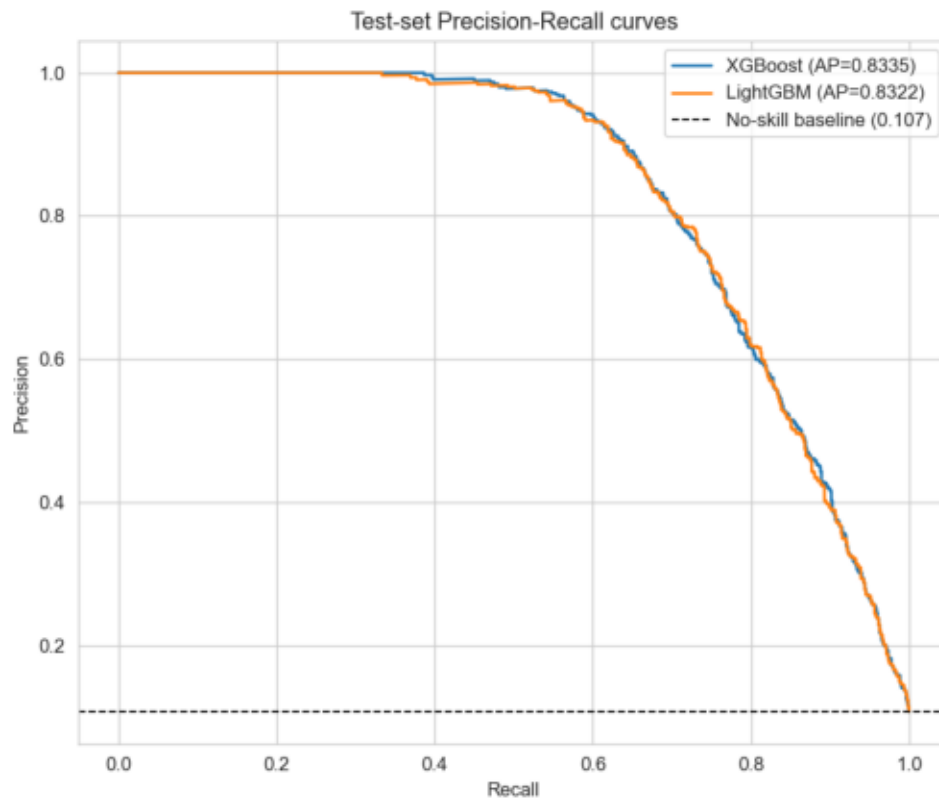
- XGBoost, threshold 0.338 (F1-optimal): Precision 0.832, Recall 0.685, F1 0.752 — best F1 of any model, any threshold, in the entire project.
- XGBoost, threshold 0.171 (F2-optimal, recall-leaning alternative): Precision 0.648, Recall 0.785 — still far better precision than any Batch 2 sampling result at similar recall (0.467 at 0.817 recall).
- LightGBM, threshold 0.378 (F1-optimal): Precision 0.850, Recall 0.670, F1 0.750 — a very close second, and a reasonable alternative if training/inference speed matters more than the last fraction of a percentage point.

Final Test-Set Confusion Matrices (F1-Optimal Thresholds)

Final test-set confusion matrices (F1-optimal thresholds)



Final Test-Set PR and ROC Curves



Negative Findings — Reported in Full

Per the project's experiment-tracking requirement, every change that did NOT improve performance is recorded here explicitly, not hidden:

- CatBoost (Batch 1): tied with XGBoost/LightGBM on PR-AUC but took 22-28x longer to train (1,519s vs 55-69s). Dropped from further tuning on cost/benefit grounds.
- Class weighting, SMOTE, random over/under-sampling, SMOTEENN, SMOTETomek (Batch 2): increased recall but decreased precision enough that PR-AUC fell for every technique, on every one of 5 models tested. None beat the no-sampling baseline.
- Redundant-feature removal (Batch 2): no meaningful effect for XGBoost, LightGBM, Logistic Regression, or MLP (noise-level differences, ± 0.002 -0.003 PR-AUC). Actively hurt Random Forest (-0.018 PR-AUC for the most aggressive reduction).
- Regularization refinement (Batch 3, Part A): technically improved CV PR-AUC for XGBoost, LightGBM, and Logistic Regression, but by margins ($+0.0002$ to $+0.0007$) too small to be practically meaningful — Batch 1's original search had already found close to the achievable ceiling for these architectures on this data.
- Decision Tree, SVM, Naive Bayes (Batch 1): consistently behind the leaders throughout; Naive Bayes especially (PR-AUC 0.44) due to its independence/Gaussian assumptions being violated by this dataset's skewed, correlated financial features. Not carried past Batch 1.

Final Recommendation

XGBoost, full feature set, no sampling, threshold = 0.338 (F1-optimal).

Test PR-AUC 0.8335 · Test ROC-AUC 0.9469 · Precision 0.832 · Recall 0.685 · F1 0.752

Why XGBoost over LightGBM:

- Marginally better on every metric that matters (PR-AUC, F1, precision, recall) on the untouched test set — not a large margin, but a consistent one.
- Both are practically equivalent in training cost (~55-80s), so there's no efficiency reason to prefer LightGBM here; LightGBM remains a reasonable substitute if needed.

Why this threshold, not 0.5:

- The default 0.5 threshold is an arbitrary convention, not a property of the data. 0.338 is the threshold that best balances precision and recall (F1) as measured honestly via cross-validation on the training set, confirmed on validation and again on the untouched test set.
- If the business weighs missing fraud as costlier than a false accusation, threshold 0.171 (F2-optimal) trades down to 0.648 precision for 0.785 recall — still comfortably ahead of anything sampling achieved at similar recall.

In practical terms:

Out of every 100 businesses this model flags as fraud, about 83 actually are. Of all the fraud that exists, it catches about 69%. That is a substantial, practically useful screening tool, directly tunable toward fewer false accusations or more fraud caught depending on business tolerance for each type of error.